

```
Adding group 'docker' (GID
116)
Done.
Builds finished. The artifacts
of successful builds are:
AMIs were created:
us-east-1: ami-
05f11967dcc85276f
```

As you can see, both the shell provisioners are run before building the AMI. Also, it can be found from the output that Packer logged into the instance using SSH, and ran the scripts to install *docker*s.

We can verify if the provisioning is done properly. For that, log in to the AWS console, launch an EC2 instance using the newly built AMI with a key pair, and note down the public IP address. Download the key pair as a *.pem* file onto the local machine.

Run the following commands. Replace the *.pem* file and IP address appropriately. In my case, the *.pem* file is named *glarimy*, and the IP address is 52.23.169.102.

```
chmod 400 glarimy.pem
ssh -i glarimy.pem ubuntu@52.23.169.102
sudo docker images
```

Notice that the newly launched EC2 instance is equipped with a Docker engine and you can interact with it.

Handling the artifacts

Well! We have covered good ground. We now know how to select a source AMI, how to clone it, how to provision it, and how to create a new AMI. In the Packer jargon, the newly generated files such as AMIs are referred to as artifacts. Packer also helps in handling these artifacts. For instance, Packer can help you in registering an AMI with a different registry, or move to a different place, etc. Such artifact handling is achieved using *post processors*.

In the following example, it simply creates a file on the local machine with the details of the newly created artifact.

Add a *manifest post-processor* to the *build* block in the *ami.pkr.hcl*. This entry directs the packer to create a file named 'glarimy-ubuntu.json' with the manifest details of the newly built AMI artifact.

```
build {
  ...

  post-processor "manifest" {
```

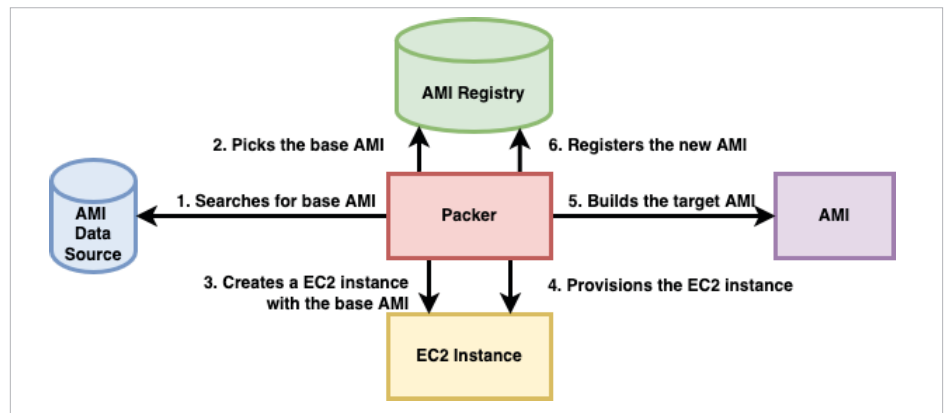


Figure 4: Packer build process

```
output = "glarimy-ubuntu.json"
}
```

Running the updated template generated the following manifest in my case, in the *glarimy-ubuntu.json*.

```
{
  "builds": [{
    "name": "glarimy-ubuntu",
    "builder_type": "amazon-eks",
    "build_time": 1683021300,
    "files": null,
    "artifact_id": "us-east-1:ami-0903822880fb16d20",
    "packer_run_uuid": "9f755e3c-7321-679d-8f95-f57256125039",
    "custom_data": null
  }],
  "last_run_uuid": "9f755e3c-7321-679d-8f95-f57256125039"
}
```

In this article, we have seen the usage of Packer in building an AWS AMI. The source code used in this article can be found at <https://bitbucket.org/glarimy/glarimy-university/src/master/glarimy-packer>. The whole process is summarised in Figure 4.

HashiCorp Packer is a powerful and easy-to-use tool to build identical images targeting multiple platforms. Using similar steps, images can be created in Azure, Google Cloud, and Docker, to name a few. **END** 🐧

By: Krishna Mohan Koyya

The author is the founder and principal consultant at Glarimy Technology Services, Bengaluru. He has mentored and upskilled more than 250 technical teams in architecting, designing, and developing enterprise applications on-premises as well as on cloud.